

MASTER SYSTEM DOCUMENT: Theater Management System (TMS)

Document Control

Information	Details
Document Name	Master System Document (FRS & Architecture)
Project Name	Xynema - Theater Management System (TMS)
Version	1.0
Status	Final

1. FUNCTIONAL REQUIREMENT SPECIFICATION (FRS)

1.1 Overview

The Theater Management System (TMS) is a comprehensive platform designed to bridge the gap between theater owners, administrative staff, and end consumers. It facilitates online movie ticket booking, public/private event management, offline counter ticketing, and robust backend analytics.

1.2 User Roles & Access Definition

The system enforces strict Role-Based Access Control (RBAC) across four primary roles.

1.2.1 Super Admin

Role Description: The Super Admin manages the entire platform infrastructure including theater onboarding, global movie catalog, special events, platform analytics, and overall revenue tracking.

Functional Requirements:

- **Authentication:** The system shall allow secure login/logout, password reset, and enforce role-based dashboard access.
- **Theater Management:** The system shall allow viewing, approving, rejecting, or suspending theater registration requests, and viewing real-time theater performance analytics.
- **Movie Management:** The system shall allow adding, editing, and deleting movies in the global catalog, uploading media (posters/trailers), and setting global metadata (Language, Genre, Certification, Formats).
- **Event Management:** The system shall allow the creation of public events (concerts, festivals) and the approval/management of user-requested private events, including digital invitation generation.
- **Revenue & Monitoring:** The system shall display total platform revenue (ticket sales + convenience fees) and generate downloadable reports filtered by theater, movie, or date range.

1.2.2 Theater Owner

Role Description: The Theater Owner acts as a localized administrator, managing the operational logistics of their own approved theater(s).

Functional Requirements:

- **Authentication:** The system shall allow secure login via the theater portal with OTP validation.
- **Theater Profile:** The system shall allow the submission of theater registration and updates to theater details (Amenities, Address, Images, Open/Closed status).

- **Infrastructure Setup:** The system shall allow adding screens, defining screen specifications (IMAX, Dolby Atmos), and utilizing an interactive grid builder to map rows/columns, categorize seats (VIP, Gold), and set base pricing.
- **Show Scheduling:** The system shall allow scheduling shows by linking global movies to specific screens, applying pricing modifiers, and preventing overlapping shows on the same screen.
- **Staff Management:** The system shall allow creating, activating, and deactivating counter staff accounts.
- **Analytics:** The system shall display localized dashboards for daily/weekly revenue and screen occupancy rates.

1.2.3 Staff (Counter Operator)

Role Description: Staff members handle day-to-day ticketing operations, offline box office bookings, and guest check-ins.

Functional Requirements:

- **Authentication:** The system shall allow login with owner-provided credentials, strictly restricted to their assigned theater.
- **Box Office Booking:** The system shall allow staff to select movies/shows, view the real-time seat layout, select seats for walk-in customers, process offline payments (Cash/Card/UPI), and print physical tickets.
- **Booking Management:** The system shall allow staff to view the daily booking list, query bookings by ID/Phone, reprint tickets, and process counter cancellations (pre-showtime).
- **Guest Check-In:** The system shall allow staff to validate and check in guests using QR code scanners.
- **Restrictions:** Staff shall NOT add/modify movies, schedule shows, modify seat layouts, or access revenue analytics.

1.2.4 End User (Customer)

Role Description: The End User leverages the web or mobile application to discover entertainment options and book tickets securely.

Functional Requirements:

- **Authentication:** The system shall allow account registration, secure login (OTP/Email), password recovery, and profile management.
- **Discovery Engine:** The system shall allow searching by location, movie, date, and language, displaying detailed movie synopsis, show timings, and pricing.
- **Seat Selection & Locking:** The system shall allow interactive seat selection. Selected seats shall be temporarily locked (e.g., 5 minutes) to prevent concurrency collisions, releasing automatically upon timeout.
- **Payment & Booking:** The system shall route users through a secure payment gateway, confirming the booking and permanently allotting seats *only* upon successful payment authorization.
- **Ticket Management:** The system shall generate digital E-Tickets (QR Code), allow viewing of booking history, and handle online cancellations and automated prorated refunds.

2. ROLE-BASED ACCESS CONTROL (RBAC) SUMMARY

Feature	Super Admin	Theater Owner	Staff	Customer
Approve/Manage Theaters	Yes	No	No	No
View Platform Revenue	Yes	No	No	No
Create Global Movies	Yes	No	No	No
Configure Screens & Layouts	No	Yes	No	No
Schedule Shows	No	Yes	No	No
Manage Theater Staff	No	Yes	No	No
View Theater Revenue	No	Yes	No	No
Counter/Offline Booking	No	Yes	Yes	No
Event Validations / Check-in	No	Yes	Yes	No
Online Booking & Payment	No	No	No	Yes

3. SYSTEM ARCHITECTURE & DATABASE DESIGN

3.1 Technology Stack

The platform utilizes the MERN stack, augmented with modern tooling for real-time performance and asynchronous processing:

- **Frontend:** React.js (with Vite), Tailwind CSS, Recharts (analytics), React Router.
- **Backend:** Node.js with Express.js.
- **Database:** MongoDB (NoSQL) hosted on MongoDB Atlas, modeled utilizing Mongoose ODM.
- **Background Jobs:** BullMQ paired with Redis.
- **Notification :** RabbitMQ email notifications and invitation generation.
- **Cloud Storage:** Cloudinary or AWS S3 (for media and KYC documents).

3.2 Core Component Architecture

- **API Gateway / Controller Layer:** Express routes validate JWT tokens via `auth.middleware.js` before passing sanitized standard input to specific controllers. Data is strictly isolated by `theaterId` for Owners and Staff.
- **Real-Time Seat Locking:** A hybrid approach using MongoDB TTL (Time-To-Live) indexes or Redis temporarily locks selected seats during the checkout timer.
- **Analytics Engine:** Complex MongoDB aggregation pipelines (`$group`, `$match`, `$project`) are utilized to calculate metrics like revenue and occupancy.

3.3 Database Schema Design (MongoDB)

Authentication Models: * SuperAdmin: `_id`, `email`, `passwordHash`, `name`

- **TheaterOwner:** `_id`, `email`, `phone`, `passwordHash`, `status`, `kycDetails`
- **Staff:** `_id`, `theaterId` (ref), `ownerId` (ref), `role`, `email`, `passwordHash`, `status`

- **User:** _id, name, email, phone, bookingHistory [ref]

Core Theater Models:

- **Theater:** ownerId (ref), name, location (GeoJSON 2dsphere), status
- **Screen:** theaterId (ref), layout (rows, columns), seatCategories, totalCapacity

Operational & Transactional Models:

- **Show:** movieId (ref), theaterId (ref), screenId (ref), showDate, startTime, status
- **Booking:** userId (ref), showId (ref), seats [array], totalAmount, status (confirmed/pending), paymentId, bookedBy

3.4 API Architecture Design

The RESTful API is organized by domain and secured via JWT Bearer tokens:

- /api/superadmin/*: Movie CRUD, Global Analytics, Theater Approvals.
- /api/theater/*: Screen Management, Show Scheduling, Staff Management.
- /api/movies/*: Public endpoints for content discovery.
- /api/booking/*: Seat locking, payment initiation, and webhook receivers for payment success.

3.5 Security & Performance Considerations

- **Geospatial Indexing:** Utilizes MongoDB's 2dsphere indexes on Theater coordinates to enable highly performant \$near queries for "Cinemas near me" features.
 - **Input Sanitization:** Strict validation using express-validator to prevent injection attacks.
 - **Rate Limiting:** IP-based rate limiting via rateLimiter.js to restrict brute-force attempts on sensitive endpoints (login, OTP).
-